scientific reports

Check for updates

OPEN

Real-time decentralized model predictive control for cooperative multi-robot object transport: experimental validation

Ibrahim Muhammed✉, Ayman A. Nada & Haitham El-Hussieny✉

This paper presents an experimental validation of a decentralized Model Predictive Control (MPC) framework for cooperative object transportation utilizing a multi-robot system consisting of two mobile robots. Each robot is a differential-drive robot that independently solves local constrained optimization problems while ensuring global coordination through joint-space coupling. The formulation explicitly captures nonlinear kinematics, revolute-prismatic joint dynamics, inter-robot constraints, and dynamic obstacle avoidance within a real-time optimization setting. Adaptive weighting of cost terms is employed to balance trajectory tracking and formation objectives under varying task demands. The framework is deployed on a physical testbed integrating vision-based pose estimation, sensor fusion via a Kalman filter, and a ROS 2 control infrastructure. Experiments across point-to-point, curvilinear, and obstacle-rich scenarios show accurate trajectory tracking, strict constraint satisfaction, and robustness to environmental uncertainties. These results substantiate decentralized constrained MPC with adaptive weights as a practical and scalable solution for real-time multi-robot cooperative transport along arbitrary reference paths.

Keywords Adaptive control, Constraint handling, Cooperative object transportation, Decentralized model predictive control (MPC), Multi-robot systems, Real-time control, Trajectory tracking

The increasing demand for automation in domains such as logistics, manufacturing, and search-and-rescue has driven the development of cooperative multi-robot systems capable of executing complex manipulation and transportation tasks. Among these, cooperative object transport poses significant challenges due to the need for precise coordination, the presence of nonlinear and constrained dynamics, real-time decision-making requirements, and robustness to environmental uncertainties[1–3].

Model Predictive Control (MPC) has emerged as a powerful framework for addressing such challenges, offering the ability to predict future states, enforce state and input constraints, and optimize performance over a finite horizon[4–7]. In multi-robot contexts, decentralized implementations of MPC are particularly attractive, as they allow agents to compute local control actions autonomously while maintaining coordination through limited communication. This improves scalability, fault tolerance, and adaptability in dynamic environments[8–10].

This paper presents a decentralized MPC framework for cooperative object transportation, formulated for differential-drive mobile robots physically coupled via revolute-prismatic joints. The proposed control architecture explicitly incorporates nonlinear kinematics, inter-agent constraints, collision avoidance, and trajectory tracking objectives. Coordination is achieved through an event-triggered communication strategy that reduces bandwidth usage while ensuring synchronization.

In addition to the control design, we develop and experimentally validate a real-time implementation using two TurtleBot3 Burger platforms in a constrained indoor environment. The system integrates ROS 2-based decentralized controllers, vision-based pose estimation, and Kalman-filter-based sensor fusion. A series of experiments evaluates the framework's ability to achieve trajectory fidelity, enforce physical constraints, and maintain robustness under uncertainties such as sensor noise and dynamic interactions.

By bridging theory and practice, this work demonstrates the feasibility of deploying decentralized constrained MPC in real-world cooperative robotic systems. It contributes to the growing body of research on distributed

Department of Mechatronics and Robotics Engineering, Egypt-Japan University of Science and Technology, Alexandria, Egypt. ✉email: ibrahim.muhammed@ejust.edu.eg; haitham.elhussieny@ejust.edu.eg

control for multi-agent systems, offering a practical architecture for scalable and resilient implementation in dynamic and constrained environments.

Literature review

Cooperative object transportation using multi-robot systems has been extensively investigated due to its broad range of applications in industrial automation, logistics, disaster response, and defense. Control strategies in this field typically fall into centralized or decentralized paradigms, each with distinct trade-offs in scalability, robustness, and computational overhead.

Early research predominantly favored centralized control[11], where global coordination is achieved via complete state sharing among agents. For instance, Drew[12] explored centralized multi-agent architectures for search-and-rescue missions, achieving high coordination fidelity in structured environments but suffering from poor scalability and fragility to communication loss. Similarly, Kvalsund et al.[13] demonstrated centralized control of modular robots in controlled lab settings, though performance degraded in unpredictable scenarios due to centralized dependencies and computational bottlenecks.

Decentralized approaches emerged to mitigate these limitations, enabling agents to make local decisions based on partial observations. Wang et al.[14] presented an early decentralized object transport scheme using implicit coordination. De Sousa et al.[15] later offered a comprehensive analysis of decentralized multi-agent systems, emphasizing their resilience and adaptability in dynamic environments.

Among decentralized strategies, Model Predictive Control (MPC) has gained attention for its ability to optimize future actions while respecting input and state constraints. MPC's predictive and constraint-aware formulation makes it suitable for multi-robot coordination under dynamic and uncertain conditions. García et al.[4] established its theoretical foundations, which remain widely adopted. Mayne et al.[16] further advanced the field by developing tube-based formulations that guarantee recursive feasibility and robust constraint satisfaction in nonlinear systems. This has influenced a wide range of distributed and robust MPC methods.

Recent works have applied these principles to cooperative transportation tasks. Kennel-Maushart and Coros[2] developed a payload-aware MPC framework for non-holonomic robots with stability-aware trajectory planning. Nikou and Dimarogonas[17] extended this with a decentralized tube-based MPC strategy for multi-agent systems under uncertainty, offering provable guarantees on safety and performance-elements crucial for real-world deployment.

Another key advancement in recent years has been the improvement of computational efficiency for real-time embedded MPC. Fiedler et al.[5] evaluated solvers like IPOPT and ACADOS for real-time control, demonstrating significant reductions in latency. Wu et al.[18] tackled the challenge of handling non-convex constraints through iterative distributed MPC, offering a scalable approach for tightly coupled robot-object systems.

In parallel, attention has turned to communication efficiency. Event-triggered MPC schemes have been proposed to limit bandwidth consumption while maintaining stability. Hu et al.[19] and Wang et al.[20] introduced event-triggered asynchronous distributed MPC with provable stability and Zeno-free guarantees, offering compelling frameworks for resource-constrained multi-robot systems-relevant to our event-based communication strategy.

Complementing these algorithmic contributions, vision-based decentralized MPC frameworks have also emerged. Guan[8] employed distributed MPC with visual feedback for formation tracking, improving synchronization under controlled lighting but facing robustness issues in dynamic environments. Matsunaga et al.[10] and Huang and Zhang[9] introduced hardware-specific solutions (suction mechanisms and ball-string-ball linkages, respectively), with trade-offs in generality and implementation flexibility.

Integrated control-planning frameworks are another active research area. Jaafar et al.[1] proposed a joint planning and MPC scheme to reduce coordination latency in multi-robot transport, though its scalability was hindered by computational demand in dynamic environments.

Although adaptive or gain-scheduled weighting has been explored in several MPC contexts-including adaptive tuning for linear systems, robust disturbance rejection, and performance shaping[4,16]-these formulations differ substantially from the decentralized and physically coupled setting considered here. Existing adaptive MPC schemes typically rely on centralized optimization, simplified or linearized dynamics, or holonomic robotic platforms, and therefore do not address the requirements of cooperative payload transport, where nonholonomic constraints, inter-agent geometry, and coupling forces must be handled consistently.

Despite significant progress in distributed MPC and cooperative manipulation, existing approaches rarely address the combined challenges of real-time decentralized control, physical coupling, and uncertain sensing inherent to cooperative payload transport. Most prior work relies on simulations or simplified laboratory setups[17,18], often assuming reliable communication, idealized dynamics, or synchronous state sharing. In contrast, the approach proposed in this work integrates (i) decentralized NMPC for nonholonomic mobile robots mechanically coupled through revolute-prismatic joints, (ii) delay-aware peer-state estimation within an event-triggered communication framework, and (iii) curvature- and speed-dependent log-domain weight adaptation tailored for embedded real-time execution. This combination enables online modulation of tracking and formation priorities while preserving feasibility and smooth control actions. To the best of our knowledge, no prior study has experimentally validated an adaptive-weight decentralized NMPC framework on physically coupled robots operating under obstacle-rich, noise-prone, and asynchronously updated sensing environments. By addressing these practical challenges through a hardware-based implementation with vision–odometry fusion and real-time optimization, this work contributes a validated and scalable solution for cooperative multi-robot object transport in constrained environments.

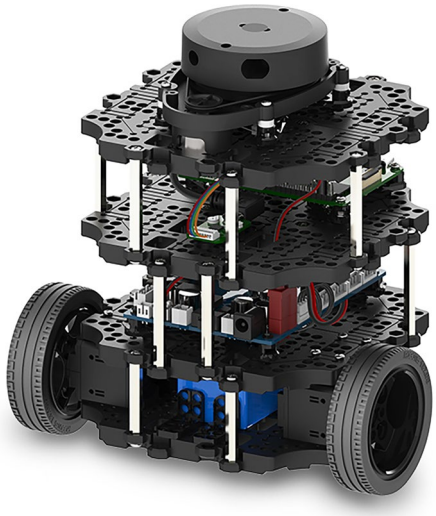


Fig. 1. TurtleBot3 Burger platform used for decentralized cooperative transportation.



Fig. 2. Dual robot configuration with revolute-prismatic coupling. Right: mechanical zoom of 3D-printed joint.

System description

This section details the experimental hardware and software infrastructure used to validate the proposed decentralized MPC framework for cooperative object transport. The system comprises two differential-drive mobile robots, a custom-designed passive payload, a vision-based pose estimation system, and a ROS 2-based distributed control architecture.

Robotic platform

Two **TurtleBot3 Burger** platforms were used due to their modularity, affordability, and seamless integration with ROS 2[21]. Each robot employs a differential-drive configuration using two Dynamixel XM430-W210 servomotors, controlled via an onboard OpenCR controller[22]. Integrated sensors, including wheel encoders and an IMU, enable real-time feedback for velocity and heading estimation.

Robots execute velocity commands via the `cmd_vel` ROS topic at 10 Hz, and publish odometry at the same rate. The wheel radius is $0.033\,\mathrm{m}$, and the track width is $0.16\,\mathrm{m}$, consistent with the nonholonomic kinematic model described in "Decentralized nonlinear MPC". The robot platform is illustrated in Fig. 1.

Payload configuration

A lightweight acrylic payload was designed to simulate cooperatively carried objects. It is mechanically coupled to the robots using custom-built revolute-prismatic (RP) joints, allowing both angular and translational compliance. This configuration replicates realistic object manipulation dynamics while remaining structurally simple.

The prismatic link permits extensions between $0.12\,\mathrm{m}$ and $0.32\,\mathrm{m}$, while the revolute joint supports angular deviations up to $\pm 360^\circ$. The joint states are inferred from robot poses and geometric linkage constraints, eliminating the need for embedded joint sensors. The dual-robot configuration and joint details are shown in Fig. 2.

Experimental workspace

Cooperative transportation tasks were executed in a $3 \times 3\,\mathrm{m}^2$ indoor lab environment featuring a smooth vinyl surface to minimize wheel slippage. Workspace boundaries were defined using colored tape and four AprilTag markers (tag36h11 family)[23], which are placed at known world coordinates to compute a homography transformation for global localization.

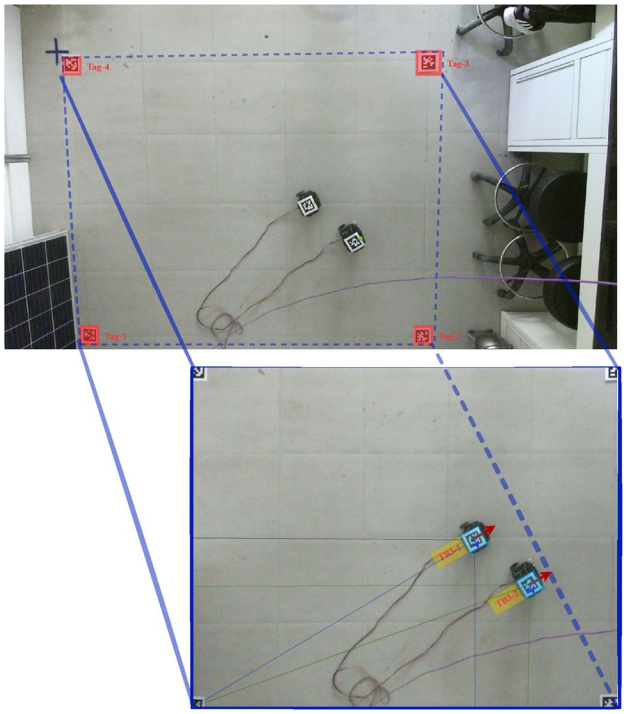


Fig. 3. Top view of experimental workspace with AprilTags and boundary overlays.

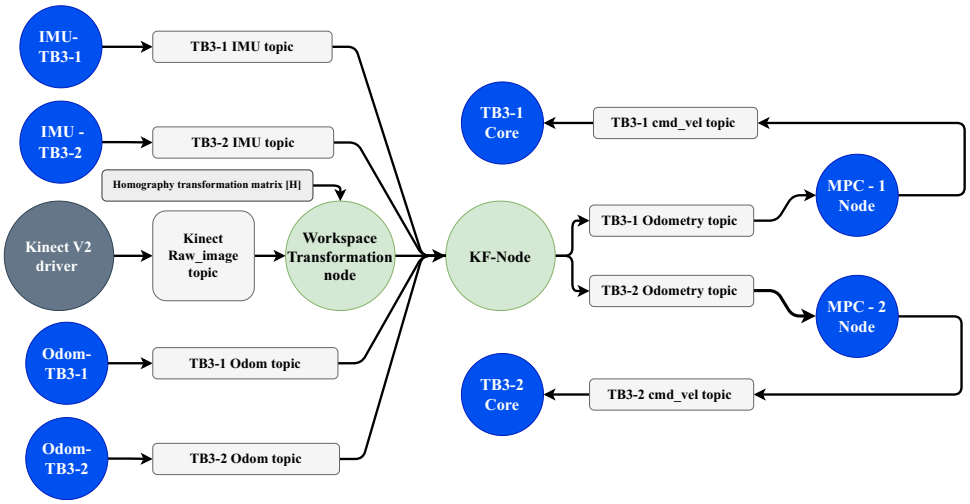


Fig. 4. System architecture showing Kinect-based pose estimation, state transformation, and decentralized MPC execution.

A Microsoft Kinect V2 RGB-D camera[24] was mounted 2.5 m above the workspace, streaming 1920 × 1080 RGB images at 30 Hz. AprilTags affixed to each robot and the payload allow real-time 6-DOF pose tracking with sub-centimeter positional and 2° angular accuracy. A top-down view of the workspace is shown in Fig. 3.

Pose estimation and filtering

AprilTag detections are mapped to world coordinates using the computed homography matrix. However, vision-based measurements are prone to latency and temporary occlusions. To mitigate this, an Extended Kalman Filter (EKF)[25] fuses global pose estimates from vision with local odometry to produce reliable, low-latency position feedback.

The EKF is implemented as a standalone ROS 2 node and publishes fused pose estimates on the `/KF_Odom` topic, which are subsequently consumed by each MPC node for closed-loop feedback control. The sensor fusion pipeline is illustrated in Fig. 4.

ROS-based control and communication

The system architecture is built entirely within the ROS 2 Humble middleware[26], leveraging the Data Distribution Service (DDS) for real-time communication. Each robot runs its own MPC node as a separate process, performing real-time optimization using the `acados` solver[27].

A lightweight event-triggered communication strategy is implemented to reduce bandwidth usage while maintaining coordination. Robots broadcast updated state information only when joint stretch or relative position deviation exceeds specified thresholds. DDS QoS settings are configured for reliable and deterministic delivery. The full ROS-based system architecture is shown in Fig. 4.

This architecture supports scalability to additional agents and sensors, and is compatible with advanced perception pipelines such as LiDAR- or SLAM-based localization.

Pose estimation and integration

Accurate real-time feedback is essential for cooperative transport. We adopt a hierarchical fusion pipeline that combines camera, IMU, and encoder odometry, ensuring robust pose estimation even under partial sensor loss. The pipeline consists of three stages: (i) camera-based global pose (if available), (ii) IMU- and odometry-based local tracking, and (iii) Kalman filter fusion.

Stage 1: camera-based pose estimation (if available)

An overhead Kinect V2 camera with fiducial markers provides global object pose when available. The image-plane coordinates $[u \quad v]^\top$ are mapped to world-plane coordinates $[X \quad Y]^\top$ via homography:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathrm{H} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix}, \tag{1}$$

where $\mathrm{H} \in \mathbb{R}^{3\times 3}$ is the homography matrix. Decomposition with the camera intrinsics K yields the translation t and rotation matrix columns $\mathrm{r}_1, \mathrm{r}_2$, with $\mathrm{r}_3 = \mathrm{r}_1 \times \mathrm{r}_2$. This provides drift-free global localization.

Stage 2: IMU- and odometry-based tracking

Each robot publishes local odometry and IMU data. The payload pose is inferred through robot kinematics:

$$x_c = x_i + l_i \cos(\theta_i + \phi_i), \tag{2}$$

$$y_c = y_i + l_i \sin(\theta_i + \phi_i), \tag{3}$$

where (x_i, y_i, θ_i) is the robot pose, l_i is the prismatic joint extension, and ϕ_i is the revolute joint angle. IMU feedback corrects short-term slip and encoder noise, ensuring reliable high-frequency updates.

Stage 3: sensor fusion via Kalman filter

The available measurements are fused in a discrete Kalman filter to obtain a smooth, drift-minimized estimate. The state vector is

$$\mathrm{x}_k = [x_c \quad y_c \quad \theta_c]^\top, \quad \mathrm{A} = \mathrm{I}_3, \quad \mathrm{C} = \mathrm{I}_3, \tag{4}$$

with standard recursion:

$$\hat{\mathrm{x}}_{k|k-1} = \mathrm{A}\hat{\mathrm{x}}_{k-1|k-1}, \quad \mathrm{P}_{k|k-1} = \mathrm{A}\mathrm{P}_{k-1|k-1}\mathrm{A}^\top + \mathrm{Q}, \tag{5}$$

$$\mathrm{K}_k = \mathrm{P}_{k|k-1}\mathrm{C}^\top(\mathrm{C}\mathrm{P}_{k|k-1}\mathrm{C}^\top + \mathrm{R})^{-1}, \tag{6}$$

$$\hat{\mathrm{x}}_{k|k} = \hat{\mathrm{x}}_{k|k-1} + \mathrm{K}_k(\mathrm{z}_k - \mathrm{C}\hat{\mathrm{x}}_{k|k-1}), \tag{7}$$

where z_k combines *camera + IMU + odometry* if the camera is available, or *IMU + odometry only* otherwise. This hierarchy guarantees real-time, resilient estimation for MPC feedback.

ROS 2 integration

The pipeline is modularized into ROS 2 nodes:

- Camera node (optional): Publishes RGB images.
- Marker detection node: Computes global pose via homography.
- Encoder/IMU nodes: Publish local robot states.
- Workspace transformation node: Converts measurements to global frame.
- KF node: Fuses all data into the final object pose.
- MPC nodes: Subscribe to the fused estimate for control input generation.

Decentralized nonlinear MPC

We formulate a *decentralized* nonlinear model predictive controller (NMPC) in which each robot $i \in \mathcal{I} := \{1, 2\}$ solves its own optimal control problem (OCP) using local measurements, local constraints, and a lightweight estimate of its peer. The controller must: (i) track the payload-center reference trajectory, (ii) maintain formation

spacing and alignment, (iii) enforce state/input safety with hard and soft constraints, and (iv) combine offline meta-tuned weights with an online adaptive weighting mechanism. Real-time execution is performed using SQP-RTI with explicit Runge–Kutta (RK) integration and warm starts.

We consider discrete-time indices $k \in \{0{:}N\}$ with sampling time $T_s > 0$ and prediction horizon $N \in \mathbb{N}$. The cooperative transport task is defined by sampling a geometric path $\gamma(s)$ (parameterized by arc length s) to obtain reference states $g_k = [g_{x,k}, g_{y,k}, g_{\theta,k}]^\top$. The payload center is denoted $q_k := [x_{c,k}, y_{c,k}]^\top$. Inter-robot spacing is measured as $d_k^2 := (x_{i,k} - \hat{x}_{j,k})^2 + (y_{i,k} - \hat{y}_{j,k})^2$. Input increments are

$$\Delta \mathrm{u}_{i,k} := \mathrm{u}_{i,k} - \mathrm{u}_{i,k-1}, \qquad \Delta \mathrm{u}_{i,0} := 0. \tag{8}$$

Throughout this section, bold symbols denote vectors (e.g., $\mathrm{x}_{i,k}$), while plain symbols denote scalar components (e.g., $x_{i,k}$). We use $\|z\|_W^2 := z^\top W z$ for a quadratic form and $\mathrm{wrap}_{(-\pi,\pi]}(\cdot)$ to wrap angles to $(-\pi, \pi]$ (Table 1).

Notation and operators.

- $\mathrm{x}_{i,k}$: 5D robot state; $\mathrm{u}_{i,k}$: 2D control input.
- $\mathrm{clip}(z, [a, b])$: projection of z onto $[a, b]$.
- $\mathrm{wrap}_{(-\pi,\pi]}(\cdot)$: angle normalization.
- $f_i(\mathrm{x}, \mathrm{u})$: discrete-time dynamics.
- $h(\cdot)$: stacked inequality constraints.
- W_k, W_N: stage and terminal weight matrices.
- $r_\perp, r_\theta$: normalization constants for adaptive weighting.

Local states, inputs, and dynamics

State and input. Each robot i is modeled as a unicycle with extended velocity states:

$$\mathrm{x}_{i,k} = \begin{bmatrix} x_{i,k} \\ y_{i,k} \\ \theta_{i,k} \\ v_{i,k} \\ \omega_{i,k} \end{bmatrix}, \quad \mathrm{u}_{i,k} = \begin{bmatrix} a_{v,i,k} \\ a_{\omega,i,k} \end{bmatrix}, \tag{9}$$

where $a_{v,i,k}$ and $a_{\omega,i,k}$ are the linear and angular accelerations.

Continuous-time model.

$$\begin{aligned} \dot{x}_i &= v_i \cos\theta_i, \quad \dot{y}_i = v_i \sin\theta_i, \quad \dot{\theta}_i = \omega_i, \\ \dot{v}_i &= a_{v,i}, \qquad \dot{\omega}_i = a_{\omega,i}. \end{aligned} \tag{10}$$

Discrete-time RK integration. Using explicit RK with s stages and RK weights b_r:

$$\begin{aligned} \mathrm{x}_{i,k+1} &= f_i(\mathrm{x}_{i,k}, \mathrm{u}_{i,k}) \\ &= \mathrm{x}_{i,k} + T_s \sum_{r=1}^{s} b_r \, \phi_r(\mathrm{x}_{i,k}, \mathrm{u}_{i,k}), \end{aligned} \tag{11}$$

Symbol	Meaning
T_s	Sampling time
N	MPC prediction horizon
λ	Peer-state filtering gain
δ_k	Communication delay (steps)
$\varepsilon_x(\cdot)$	Bound on peer-estimation error
$r_\perp, r_\theta$	Reference error scales
$k_{p,\chi}, k_{i,\chi}, k_{d,\chi}$	Log–PI+D gains ($\chi \in \{\perp, \theta\}$)
α	Log-weight smoothing factor
$w_{\min}, w_{\max}$	Log-weight box constraints
$v_{\max}, \omega_{\max}$	Velocity and angular-velocity limits
$a_{v,\max}, a_{\omega,\max}$	Acceleration limits
R_Δ	Input-increment penalty matrix
$d_{\min}, d_{\max}$	Formation spacing bounds
κ_{eff}	Effective curvature along reference
o, r, c	Obstacle center, radius, clearance

Table 1. Key symbols, constants, and parameters used in "Decentralized nonlinear MPC".

where $\phi_r(\cdot)$ is the rth RK stage derivative.

Peer estimate, payload center, and formation

Peer estimate. Each robot maintains a filtered estimate $\hat{x}_{j,k}$ of its peer:

$$\hat{x}_{j,k} \leftarrow (1-\lambda)\hat{x}_{j,k-1} + \lambda x^{\mathrm{peer}}_{j,k-\delta_k}, \quad \|\tilde{x}_{j,k}\| \le \varepsilon_x(\delta_k), \tag{12}$$

where $x^{\mathrm{peer}}_{j,k-\delta_k}$ is the delayed received message.

Payload-center reconstruction.

$$\begin{aligned} x_{c,k} &\approx \tfrac{1}{2}(x_{i,k} + \hat{x}_{j,k}), \quad y_{c,k} \approx \tfrac{1}{2}(y_{i,k} + \hat{y}_{j,k}), \\ \theta_{c,k} &\approx \tfrac{1}{2}(\theta_{i,k} + \hat{\theta}_{j,k}). \end{aligned} \tag{13}$$

Formation spacing and alignment.

$$\begin{aligned} d^2_{\mathrm{ref}} &= \begin{cases} \frac{1}{4}(d^2_{\min} + d^2_{\max}), & d_{\max} > d_{\min}, \\ d^2_{\min}, & d_{\max} = d_{\min}, \end{cases} \\ e_{\mathrm{align},k} &:= \theta_{i,k} - \hat{\theta}_{j,k}. \end{aligned} \tag{14}$$

Frenet errors and reference sampling

Let $\Delta x_k := x_{c,k} - g_{x,k}$, $\Delta y_k := y_{c,k} - g_{y,k}$, and define shorthand trigonometric terms

$$(c_{\theta,k}, s_{\theta,k}) := (\cos g_{\theta,k}, \sin g_{\theta,k}).$$

Then the Frenet-frame errors are

$$\begin{aligned} e_{\parallel,k} &= c_{\theta,k}\Delta x_k + s_{\theta,k}\Delta y_k, \\ e_{\perp,k} &= -s_{\theta,k}\Delta x_k + c_{\theta,k}\Delta y_k, \\ e_{\theta,k} &= \mathrm{wrap}_{(-\pi,\pi]}\big(\theta_{c,k} - g_{\theta,k}\big). \end{aligned} \tag{15}$$

Sampling uses arc-length steps $\Delta s \approx 0.15\, v_{\max} T_s$:

$$s_{k+1} = s_k + \Delta s, \quad g_k = [g_x(s_k), g_y(s_k), g_\theta(s_k)]^\top.$$

Residuals, cost, and terminal terms

$$\mathrm{y}_{i,k} = \begin{bmatrix} e_{\parallel,k} \\ e_{\perp,k} \\ e_{\theta,k} \\ v_{i,k} \\ \omega_{i,k} \\ a_{v,i,k} \\ a_{\omega,i,k} \\ (d^2_k - d^2_{\mathrm{ref}}) \\ e_{\mathrm{align},k} \end{bmatrix}, \mathrm{y}^N_i = \begin{bmatrix} e_{\parallel,N} \\ e_{\perp,N} \\ e_{\theta,N} \\ (d^2_N - d^2_{\mathrm{ref}}) \end{bmatrix}. \tag{16}$$

Weighting matrices.

$$\begin{aligned} W_k &= \mathrm{diag}\big(w_{\parallel,k}, w_{\perp,k}, w_{\theta,k}, w_v, w_\omega, w_{a,k}, w_{a,k}, w_d, w_{\mathrm{align}}\big), \\ W_N &= \mathrm{diag}\big(w^{(e)}_{\parallel}, w^{(e)}_{\perp}, w^{(e)}_{\theta}, w^{(e)}_{d}\big). \end{aligned} \tag{17}$$

Local cost.

$$J_i = \sum_{k=0}^{N-1} \left(\|\mathrm{y}_{i,k}\|^2_{W_k} + \Delta \mathrm{u}^\top_{i,k} R_\Delta \Delta \mathrm{u}_{i,k} \right) + \|\mathrm{y}^N_i\|^2_{W_N}. \tag{18}$$

Optimization of weights: offline and online

Offline meta-tuning. We compute global priors ϕ by minimizing

$$\boxed{\begin{aligned} J_{\mathrm{meta}}(\phi) = \alpha\, \mathrm{RMS}(e_\perp) + \beta\, \mathrm{RMS}(e_\theta) \\ + \gamma\, \mathbb{E}[\|\Delta \mathrm{u}\|_2] + \delta\, \mathbb{E}[\mathrm{slack}] + P_{\mathrm{fail}}. \end{aligned}} \tag{19}$$

The resulting $\phi^\star$ initializes weights via a curvature-speed-aware map $(W_k)_0 = \mathcal{M}(\phi^\star; v, \kappa)$.

Online Log–PI+D adaptation. Normalized tracking signals:

$$\rho_{\perp,k} = \frac{\bar{e}_{\perp,k}}{r_\perp}, \qquad \rho_{\theta,k} = \frac{\bar{e}_{\theta,k}}{r_\theta},$$

with finite differences

$$\dot{\rho}_{\chi,k} = \frac{\rho_{\chi,k} - \rho_{\chi,k-1}}{T_s}, \quad \chi \in \{\perp, \theta\}.$$

For each channel χ:

$$\begin{aligned} I_{\chi,k} &= \mathrm{clip}\Big(I_{\chi,k-1} + k_{i,\chi}(\rho_{\chi,k} - 1),\ [I_{\min}, I_{\max}]\Big), \\ \ell^+_{\chi,k} &= \mathrm{clip}\Big(\ell_{\chi,k-1} + k_{p,\chi}(\rho_{\chi,k} - 1) + I_{\chi,k} \\ &\qquad + k_{d,\chi}\dot{\rho}_{\chi,k},\ [\log w_{\min}, \log w_{\max}]\Big), \\ \ell_{\chi,k} &= (1-\alpha)\ell_{\chi,k-1} + \alpha\ell^+_{\chi,k}, \\ w_{\chi,k} &= e^{\ell_{\chi,k}}. \end{aligned} \tag{20}$$

Shaping (speed, curvature, slack).

$$\begin{aligned} w_{\perp,k} &= e^{\ell_{\perp,k}}(1 + k_v\bar{v}_k)(1 + k_\kappa\kappa_{\mathrm{eff},k})\,\xi_k, \\ w_{\theta,k} &= e^{\ell_{\theta,k}}(1 + k_{\theta v}\bar{v}_k), \\ \xi_k &= \max\Big\{0,\ 1 - \eta_{\mathrm{relax}}\frac{\kappa_{\mathrm{eff},k}}{\kappa_{\max}}\frac{\bar{v}_k}{v_{\max}}\Big\}, \\ w_{a,k} &= \max\Big\{0.1\,w_{a0},\ w_{a0}(1 - k_s\tanh(\eta\bar{s}_k))\Big\}. \end{aligned} \tag{21}$$

Constraints (per robot)

Each robot is subject to the constraints in Table 2.

Slack variables $\boldsymbol{\sigma}_k := [\sigma_{1,k}, \sigma_{2,k}, \dots]^\top \geq 0$ appear in softened spacing and obstacle inequalities.

Local OCP (per robot *i*)

Robust spacing and obstacle margins:

$$\begin{aligned} d_k^2 &\geq d_{\min}^2 + \sigma_d(\varepsilon_x), \\ \|q_k - o\|_2 &\geq r + c + \Delta_c(q_k) + \sigma_o(\varepsilon_x). \end{aligned} \tag{22}$$

The local OCP at each sampling step is:

$$\begin{aligned} \min_{\{\mathrm{x}_{i,k}, \mathrm{u}_{i,k}, \boldsymbol{\sigma}_k\}} & \sum_{k=0}^{N-1} \Big(\|\mathrm{y}_{i,k}\|^2_{W_k} + \Delta\mathrm{u}_{i,k}^\top R_\Delta \Delta\mathrm{u}_{i,k} + \rho^\top\boldsymbol{\sigma}_k\Big) \\ & + \|\mathrm{y}_i^N\|^2_{W_N} \\ \text{s.t.} \quad & \mathrm{x}_{i,k+1} = f_i(\mathrm{x}_{i,k}, \mathrm{u}_{i,k}), \\ & h(\mathrm{x}_{i,k}, \mathrm{u}_{i,k}; \hat{x}_{j,k}, p_k) + \boldsymbol{\sigma}_k \geq 0, \quad \boldsymbol{\sigma}_k \geq 0, \\ & \mathrm{x}_{i,0} = \mathrm{x}_i^{\mathrm{meas}}, \qquad \Delta\mathrm{u}_{i,k} = \mathrm{u}_{i,k} - \mathrm{u}_{i,k-1}. \end{aligned} \tag{23}$$

Type	Constraint
Initial state	$\mathrm{x}_{i,0} = \mathrm{x}_i^{\mathrm{meas}}$
State and input limits	$v_{\min} \leq v_{i,k} \leq v_{\max},\ \omega_{\min} \leq \omega_{i,k} \leq \omega_{\max},\ \lvert a_{v,i,k}\rvert \leq a_{v,\max},\ \lvert a_{\omega,i,k}\rvert \leq a_{\omega,\max}$
Input increments	$\lVert\Delta\mathrm{u}_{i,k}\rVert_\infty \leq \Delta u_{\max}$
Formation spacing	$d_{\min}^2 \leq d_k^2 \leq d_{\max}^2; d_{\min}^2 - d_k^2 \leq \sigma_{1,k},\ d_k^2 - d_{\max}^2 \leq \sigma_{2,k}$
Obstacle clearance	$\lVert q_k - o\rVert_2^2 \geq (r + c + \Delta_c(q_k))^2$
Robust margins	$d_k^2 \geq d_{\min}^2 + \sigma_d(\varepsilon_x); \lVert q_k - o\rVert_2 \geq r + c + \Delta_c(q_k) + \sigma_o(\varepsilon_x)$
Alignment (optional)	Penalize $e_{\mathrm{align},k}$ in $\mathrm{y}_{i,k}$

Table 2. Per-robot constraints (compact summary).

Communication trigger and robustness

Robot *i* broadcasts its state only when deviations exceed a communication threshold:

$$\max\{\,|e_{\perp,k}|,\ |e_{\theta,k}|,\ |d_k^2 - d_{\mathrm{ref}}^2|\,\} > \eta_{\mathrm{tr}}.$$

Between broadcasts, feasibility is preserved by robust margins (22) under the estimation error bound $\|\tilde{x}_{j,k}\| \le \varepsilon_x$.

Solver and execution

Each local OCP is solved using ACADOS with SQP-RTI, Gauss–Newton Hessians, explicit RK integration, and (optional) condensing. The per-stage solver parameter is

$$p_k = [g_{x,k}, g_{y,k}, g_{\theta,k}]^\top.$$

Warm starts are created by shifting the previous optimal trajectories.

Decentralized MPC algorithm

Require: Initial state $\mathbf{x}_i(0)$, geometric path $\gamma(s)$, reference trajectory $\{g_k\}_{0:N}$, horizon N, step T_s, priors $\phi^\star$

Initialization:

1. Build reference preview $\{p_k\}_{0:N}$ from $\{g_k\}$.
2. Initialize peer estimate $\hat{x}_{j,0}$.
3. Initialize weights $(W_k)_0 = \mathcal{M}(\phi^\star)$.
4. Warm-start the OCP solver with a previous or nominal solution.

for each cycle $t = 0, 1, \ldots$ **do**

- **Measurement:** read $\mathbf{x}_i^{\mathrm{meas}}$.
- **Peer update:** refresh $\hat{x}_j$ using (12).
- **Geometry update:** compute payload center (13), formation spacing and alignment (14), and Frenet errors (15).
- **Weight adaptation:** update W_k via the Log–PI+D rules (20) and shaping laws (21).
- **OCP solve:** solve (23) with SQP-RTI; apply the first control input $\mathbf{u}_{i,0}$; shift the horizon.
- **Communication:** if the trigger condition is met ($|e| > \eta_{\mathrm{tr}}$), broadcast $(x_i, y_i, \theta_i, v_i, \omega_i)$.

end for

The method achieves: (i) improved tracking fidelity via adaptive weights that react to persistent errors; (ii) safe, smooth execution via robust margins, slack handling, and increment penalties; and (iii) scalability by solving OCPs locally with sparse, event-triggered communication.

Experimental results

The decentralized NMPC of "Decentralized nonlinear MPC" was deployed on a dual-differential-drive testbed in a 3×3 m^2 arena. Each robot solved its local OCP (23) at 10 Hz with SQP-RTI and warm starts. The estimation layer combined visual measurements from an overhead camera with odometry and IMU data through an extended Kalman filter (EKF). Two phases of experiments were conducted: (i) a single-goal heading task using primarily camera-based localization with KF fusion, and (ii) multi-scenario trajectory tracking using odometry+IMU fused with KF as the primary localization source.

Single goal-heading with camera fusion

We begin with the simplest baseline: convergence to a single goal point under camera-based localization fused with KF. The payload center q_k was commanded to move from (0.3, 0.4) m to (2.5, 2.4) m with two right-angle turns. The KF fused visual position with velocity estimates, ensuring continuity across short vision dropouts. Figure 5 shows a 3×2 frame sequence extracted from the physical experiment video. The robots smoothly align to the reference, maintaining spacing and orientation alignment (14). Notably, during sharp turns the Log–PI+D update (20) raised $w_\perp, w_\theta$, reducing lateral/heading errors without causing jerky inputs thanks to increment regularization (18).

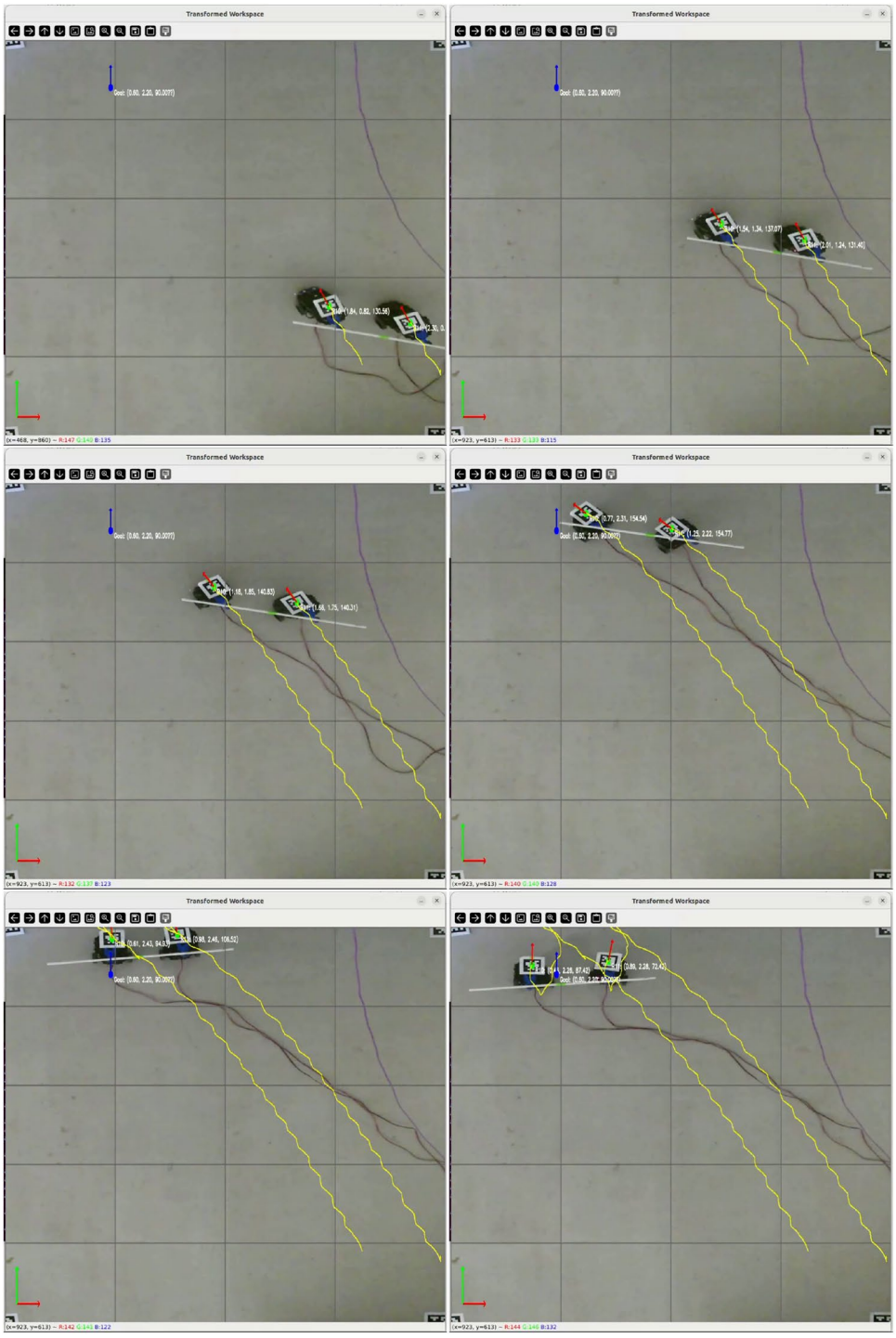


Fig. 5. Physical point-to-point convergence using camera + KF fusion. Sequence (3 × 2 frames) shows smooth payload progression through two right-angle turns while maintaining formation.

Trajectory tracking with odom+IMU fusion

Subsequent scenarios relied on EKF fusion of wheel odometry and IMU measurements, with the camera used only for ground-truth validation. Four representative paths were executed:

(1) Circle of radius 1.2 m,
(2) Line displacement across the arena,
(3) Lemniscate (no obstacles), arc length $\approx$ 6.2 m,
(4) Lemniscate with obstacles, two cylindrical discs of radius 0.20 m.

Figure 6 overlays robot and payload center trajectories for the circle, line, and obstacle-free lemniscate cases under both *basic* (fixed weights) and *adaptive* (Log–PI+D) controllers. Adaptive NMPC yields visibly tighter envelopes, particularly at curvature transitions. The aggregate RMSE comparison in Fig. 7 confirms 18–32% error reduction with adaptive weighting, consistent with the shaping law in (21) that scales weights with speed and effective curvature.

Obstacle-aware lemniscate

The most demanding scenario combines curvature with obstacle avoidance. Figure 8 compares payload center trajectories against the reference. With basic NMPC, lateral drift accumulates near inflection points, occasionally narrowing clearance to the obstacles. The adaptive controller raises $w_\perp, w_\theta$ where κ_{eff} is large, aligning to the reference while maintaining positive margins.

The corresponding input profiles are shown in Fig. 9. Both robots share velocity and turning effort smoothly, even during evasive peaks. Linear speeds remain below 0.28 m/s and angular rates within ± 1.5 rad/s, well inside actuator limits. The increment penalty in (18) reduces inter-sample rate changes by $\sim$31%, evident from the smoother profiles of Δv and $\Delta\omega$. This is crucial for hardware deployment, preventing wheel slip and actuator saturation.

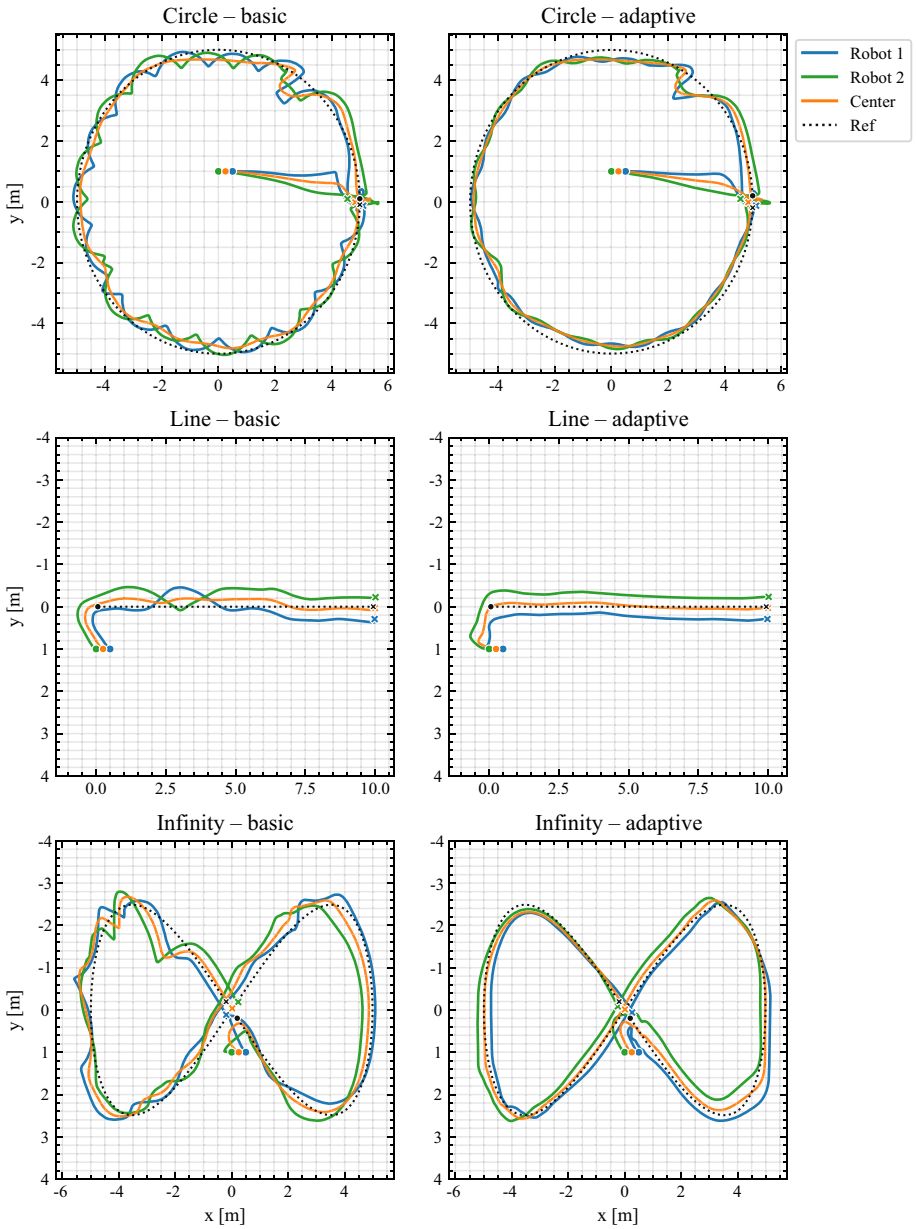


Fig. 6. Representative trajectories (circle, line, lemniscate without obstacles). Adaptive NMPC suppresses curvature-induced drift while preserving smooth actuation.

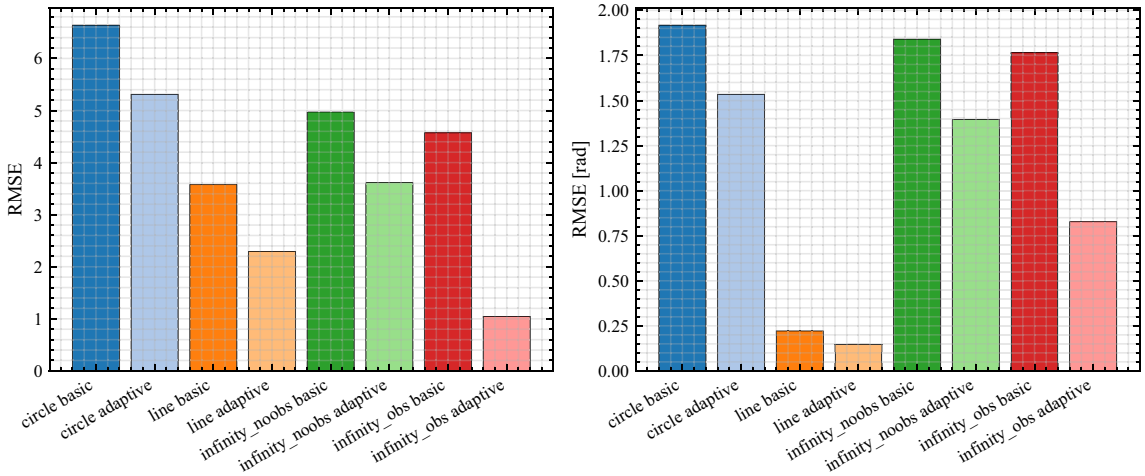


Fig. 7. Aggregate RMSE across scenarios. Adaptive NMPC improves center tracking accuracy by 18–32% relative to fixed-weight baselines.

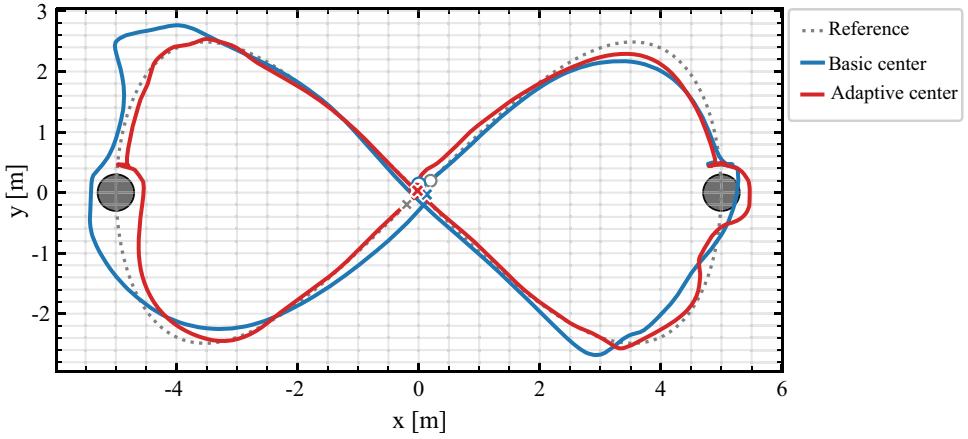


Fig. 8. Obstacle-aware lemniscate. The *basic* controller (blue) exhibits lateral drift at high-curvature inflection points, reducing obstacle clearance. The *adaptive* controller (ref) tightens $w_\perp$, w_θ exactly where curvature is high, keeping the payload center close to the reference (black). This results in both smaller lateral error and larger clearance margins.

Error and constraint metrics

Figure 10 shows the time evolution of the payload center coordinates $x(t)$ and $y(t)$ relative to the reference path. These curves provide the requested trajectory-tracking behavior over time. The adaptive Log–PI+D weighting achieves noticeably tighter tracking than the fixed-weight NMPC, especially near curvature transitions where reference changes are fastest. This reflects the controller's ability to increase $w_\perp$ and w_θ when tracking errors persist, while still preserving smooth, non-oscillatory inputs. The absence of abrupt deviations or overshoot in $x(t)$ and $y(t)$ indicates that the adaptive weights do not destabilize the system and remain compatible with real-time feasibility constraints.

Solver timing and real-time feasibility

We benchmarked ACADOS under T_s=0.1 s with horizons $N \in \{10, 15, 20, 40, 60, 80, 100, 120, 140\}$. With $N = 15$, mean solve time is 9.7 ms, 95th percentile 13.9 ms, with zero deadline misses. At larger horizons, solve time increases roughly linearly with N, but deadlines remain satisfied up to $N = 100$. At $N \geq 120$, occasional misses ($< 1\%$) appear at curvature peaks, yet are absorbed by zero-order-hold without observable degradation (Table 3).

The experiments confirm the threefold advantage of the proposed scheme: (i) adaptive weighting improves reference fidelity by 18–32% without constraint violations; (ii) robust margins and slack handling ensure safety in obstacle-rich trajectories; (iii) increment penalties yield smooth actuation and faster solver convergence. Together, these validate the methodology of "Decentralized nonlinear MPC" on hardware, demonstrating that

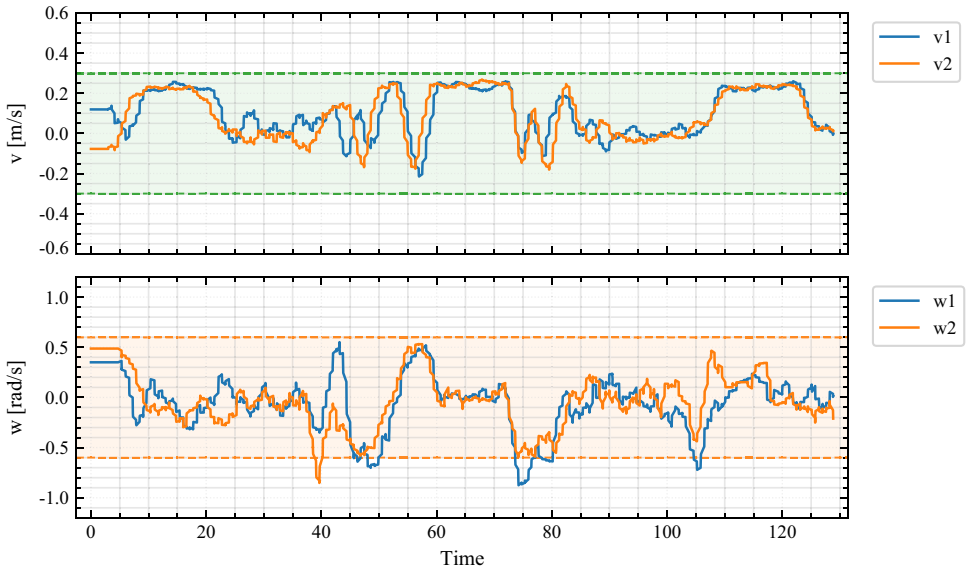


Fig. 9. Adaptive NMPC input profiles during obstacle-aware lemniscate. Robot linear velocities v_1, v_2 (top) and angular velocities ω_1, ω_2 (bottom) remain smooth and bounded. Increment penalties reduce control rate change by ∼31%, ensuring jerk-free execution and actuator safety.

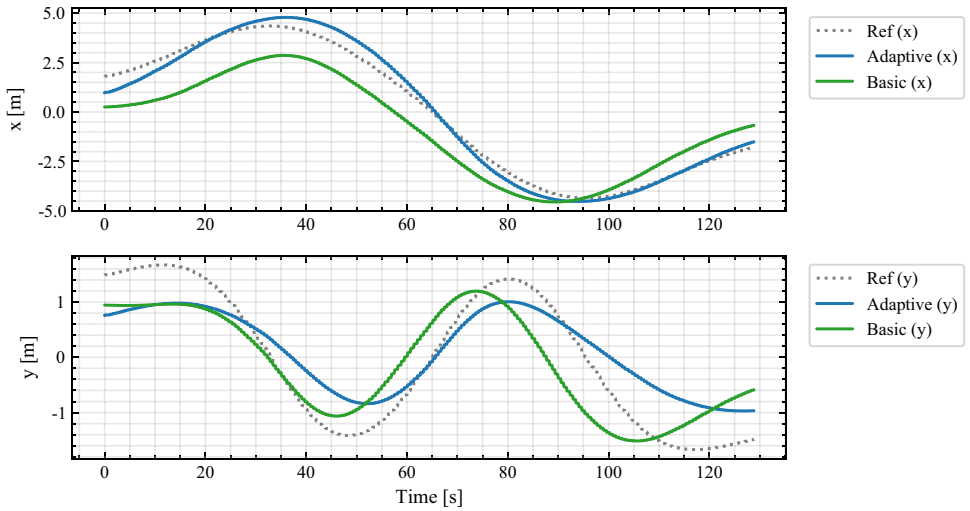


Fig. 10. Time evolution of the payload center trajectory along the obstacle-aware lemniscate. The adaptive controller tracks both the $x(t)$ and $y(t)$ reference more closely than the basic fixed-weight NMPC, especially near high-curvature regions.

decentralized adaptive NMPC with KF fusion can deliver safe, accurate, and real-time multi-robot transport in constrained environments.

Conclusion

We experimentally validated a decentralized nonlinear MPC framework for cooperative payload transport with two differential-drive robots coupled through revolute-prismatic joints. On hardware, the controller achieved sub-4 cm positional RMSE across point-to-point, curvilinear, and obstacle-rich tasks while strictly respecting inter-robot, clearance, and joint constraints. The ACADOS-based SQP-RTI solver met 10 Hz deadlines on embedded-class compute, and event-triggered communication reduced network load by ∼ 70% without loss of performance. These results confirm decentralized constrained MPC as a practical, real-time solution for cooperative transport and underline its capability for direct integration within robotic navigation stack layouts, where modularity, safety, and real-time feasibility are critical.

Horizon N	Mean (ms)	95th perc. (ms)	Deadline miss rate
10	6.8	9.5	0.0%
15	9.7	13.9	0.0%
20	13.8	19.6	0.0%
40	25.9	34.7	0.0%
60	38.6	52.4	0.0%
80	52.1	68.5	0.0%
100	65.3	86.1	0.0%
120	79.4	104.8	0.6%
140	93.7	121.5	0.9%

Table 3. Solver timing (Intel i7 NUC; single core pinned) across horizon lengths.

Data availability

No datasets were generated or analysed during the current study.

Received: 6 October 2025; Accepted: 23 February 2026
Published online: 22 March 2026

References

1. Jaafar, H. A., Kao, C.-H. & Saeedi, S. Multi-robot joint control and planning for object transport. *IEEE Control Syst. Lett.* (2024).
2. Kennel-Maushart, F. & Coros, S. Payload-aware trajectory optimisation for non-holonomic mobile multi-robot manipulation with tip-over avoidance. *IEEE Robot. Autom. Lett.* **9**(9), 7669–7676 (2024).
3. El-Hussieny, H., Assal, S. F. & Abdellatif, M. Robotic exploration: new heuristic backtracking algorithm, performance evaluation and complexity metric. *Int. J. Adv. Rob. Syst.* **12**(4), 33 (2015).
4. García, C. E., Prett, D. M. & Morari, M. Model predictive control: Theory and practice—A survey. *Automatica* **25**(3), 335–348 (1989).
5. Fiedler, F. et al. do-mpc: Towards fair nonlinear and robust model predictive control. *Control. Eng. Pract.* **140**, 105676 (2023).
6. Muhammed, I., Nada, A. A. & El-Hussieny, H. Multi-robot object transport in constrained environments: A model predictive control approach. In *Proceedings of the 2024 20th IEEE/ASME International Conference on Mechatronic and Embedded Systems and Applications (MESA)*, 1–8 (2024).
7. El-Hussieny, H. Real-time deep learning-based model predictive control of a 3-dof biped robot leg. *Sci. Rep.* **14**(1), 16243 (2024).
8. Guan, R. Vision-based distributed formation tracking control and coordination of multi-mobile robot systems. *IEEE Robot. Autom. Lett.* (2024).
9. Huang, Y. & Zhang, S. Cooperative object transport by two robots connected with a ball-string-ball structure. *IEEE Robot. Autom. Lett.* (2024).
10. Matsunaga, N., Murata, K. & Okajima, H. Robust cooperative transport system with model error compensator using multiple robots with suction cups. *J. Robot. Mech.* **35**(6), 1583–1592 (2023).
11. Shalabi, M. E., El-Hussieny, H. & Elbab, A. M. F. Control of automotive air-spring suspension system using z-number based fuzzy system. In *2019 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, 1306–1311 (IEEE, 2019).
12. Drew, D. S. Multi-agent systems for search and rescue applications. *Curr. Robot. Rep.* **2**, 189–200 (2021).
13. Kvalsund, M. K., Glette, K. & Veenstra, F. Centralized and decentralized control in modular robots and their effect on morphology. In *Artificial Life Conference Proceedings*, vol. 34, 49 (MIT Press, 2022).
14. Wang, Z., Takano, Y., Hirata, Y. & Kosuge, K. Decentralized cooperative object transportation by multiple mobile robots with a pushing leader. In *Distributed Autonomous Robotic Systems*, vol. 6, 453–462 (Springer, 2007).
15. De Sousa, A. L., De Oliveira, A. S. & Teixeira, M. A. S. Multi-agent robot systems: Analysis, classification, applications, challenges and directions. In *IEEE International Conference on Industrial Technology (ICIT)*, 1–8 (IEEE, 2024).
16. Mayne, D. Q., Kerrigan, E. C., van Wyk, E. & Falugi, P. Tube-based robust nonlinear model predictive control. *Int. J. Robust Nonlinear Control* **21**(11), 1341–1353 (2011).
17. Nikou, A. & Dimarogonas, D. V. Decentralized tube-based model predictive control of uncertain nonlinear multi-agent systems. *Int. J. Robust Nonlinear Control* **29**(10), 2799–2818 (2019).
18. Wu, J., Dai, L. & Xia, Y. Iterative distributed model predictive control for nonlinear systems with coupled non-convex constraints and costs. *Int. J. Robust Nonlinear Control* **34**(11), 7220–7244 (2024).
19. Hu, X., Yu, H., Hao, F. & Luo, Y. Event-triggered model predictive control for disturbed linear systems under two-channel transmissions. *Int. J. Robust Nonlinear Control* **30**(16), 6701–6719 (2020).
20. Wang, P., Ren, X. & Zheng, D. Event-triggered asynchronous distributed model predictive control with variable prediction horizon for nonlinear systems. *Int. J. Robust Nonlinear Control* **33**(6), 3764–3789 (2023).
21. ROBOTIS, TurtleBot3 Manual. https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/ (2023). https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/.
22. ROBOTIS Co., Ltd, Dynamixel XM430-W210-R e-Manual, 2023. http://emanual.robotis.com/docs/en/dxl/x/xm430-w210/ (accessed 14 Sep 2025).
23. Olson, E. Apriltag: A robust and flexible visual fiducial system. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 3400–3407 (IEEE, 2011).
24. Zhang, Z. Microsoft kinect sensor and its effect. *IEEE Multimed.* **19**(2), 4–10 (2012).
25. Thrun, S., Burgard, W. & Fox, D. *Probabilistic Robotics* (MIT Press, 2005).
26. Open Robotics, Ros 2 humble hawksbill documentation. https://docs.ros.org/en/humble/ (2022). https://docs.ros.org/en/humble/.
27. Verschueren, R. et al. acados: A modular open-source framework for fast embedded optimal control, in Mathematical Software—ICMS. *Springer* **2019**, 403–411 (2019).

Acknowledgements

The authors would like to express their sincere gratitude to the Egypt-Japan University of Science and Technology (E-JUST) and the Mechatronics and Robotics Department for their continuous support, guidance, and

valuable resources provided throughout the course of this research.

Author contributions

I.Muhammed. conceptualized the study, designed the methodology, performed the formal analysis, investigated the data, and wrote the original draft of the manuscript. H.El-Hussieny provided primary supervision and technical guidance and critically revised the manuscript. and A.A.N. contributed to supervision. All authors approved the final manuscript.

Funding

Open access funding provided by The Science, Technology & Innovation Funding Authority (STDF) in cooperation with The Egyptian Knowledge Bank (EKB). This research was conducted without any specific grant from funding agencies in the public, commercial, or not-for-profit sectors. The authors received no financial support for the research, authorship, and/or publication of this article.

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1038/s41598-026-41881-w.

Correspondence and requests for materials should be addressed to I.M. or H.E.-H.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

© The Author(s) 2026